WIKIPÉDIA
L'encyclopédie libre

Transaction informatique

En informatique, et particulièrement dans les bases de données, une **transaction**, telle qu'une réservation, un achat ou un paiement, est mise en œuvre via une suite d'opérations qui font passer la base de données d'un état A — antérieur à la transaction — à un état B postérieur[1] et des mécanismes permettent d'obtenir que cette suite soit à la fois *atomique*, *cohérente*, *isolée* et *durable* (ACID).

- *atomique* : la suite d'opérations est indivisible, en cas d'échec en cours d'une des opérations, la suite d'opérations doit être complètement annulée (*rollback*) quel que soit le nombre d'opérations déjà réussies.
- *cohérente* : le contenu de la base de données à la fin de la transaction doit être cohérent sans pour autant que chaque opération durant la transaction donne un contenu cohérent. Un contenu final incohérent doit entraîner l'échec et l'annulation de toutes opérations de la transaction.
- *isolée* : lorsque deux transactions A et B sont exécutées en même temps, les modifications effectuées par A ne sont ni visibles par B, ni modifiables par B tant que la transaction A n'est pas terminée et validée (*commit*).
- *durable* : Une fois validé, l'état de la base de données doit être permanent, et aucun incident technique (exemple : crash) ne doit pouvoir engendrer une annulation des opérations effectuées durant la transaction.

La majorité des systèmes de gestion de base de données *hiérarchiques* comme *relationnels* du marché permettent de réaliser des transactions atomiques, cohérentes, isolées et durables. Les systèmes *NoSQL* n'y prétendent pas[2].

Les propriétés ACID

Le concept de transaction s'appuie sur la notion de point de synchronisation (*sync point*) qui représente un état stable du système informatique considéré, en particulier de ses données.

Atomicité
: Une transaction doit s'effectuer en tout ou rien, c'est-à-dire complètement ou pas du tout ;

Cohérence
: La cohérence des données doit être assurée dans tous les cas, même dans les cas d'erreur où le système doit revenir au précédent état cohérent. La cohérence est établie par les règles fonctionnelles. Exemple : Une transaction immobilière peut durer longtemps. Informatiquement, elle sera considérée comme une procédure fonctionnelle composée de plusieurs transactions informatiques (réservation, proposition d'achat, compromis, acte notarié). Les étapes intermédiaires sont donc gérées fonctionnellement via l'état de l'objet Appartement. Même si la procédure est en cours, entre chaque étape le système reste cohérent. En cas d'abandon de la procédure (une sorte de rollback de la transaction immobilière), les règles fonctionnelles remettent l'appartement en vente. Il existe de nombreux cas de procédure qui ne peuvent pas être traités par une seule transaction informatique. Le concepteur doit spécifier les règles fonctionnelles qui pilotent les changements d'état des objets concernés et gérer manuellement l'équivalent du commit et du rollback de la procédure ;

Isolation

La transaction va travailler dans un mode isolé où elle seule peut voir les données qu'elle est en train de modifier, cela en attente d'un nouveau point de synchronisation ; le système garantit aux autres transactions, exécutées en parallèle sur le même système, une visibilité sur les données antérieures. Ceci est obtenu grâce aux verrous système posés par le SGBD. L'exemple suivant illustre une isolation plus fonctionnelle mettant en œuvre un verrouillage applicatif : Un acteur passe un ordre au vu d'un contexte. Ce contexte ne doit pas avoir changé au moment où il valide son ordre, sinon l'ordre peut perdre tout son sens. Pour éviter que le contexte ne change, il est possible de le verrouiller avant de le lire. Mais c'est souvent consommateur de ressources informatiques et gênant pour les autres surtout si la réflexion et la saisie dure plusieurs minutes. En gestion, il est généralement suffisant de simplement vérifier au moment de la mise à jour que le contexte n'a pas changé : de façon optimiste, l'isolation est vérifiée a posteriori ;

Durabilité

Lorsque la transaction est achevée, le système est dans un état stable durable, soit à l'issue d'une modification transactionnelle réussie, soit à l'issue d'un échec qui se solde par le retour à l'état stable antérieur. La durabilité est souvent en cause dans les traitements par lots (*batch*) ou asynchrones de réplication qui produisent des rejets. L'applicatif aval n'a pas le droit de remettre en cause la transaction antérieure qui a émis l'évènement, sinon cela signifierait que cette transaction n'a pas laissé le système dans un état cohérent. L'applicatif amont doit donc tout bien contrôler avant de diffuser l'information. Lorsque l'architecture fonctionnelle n'est pas pure, il est fréquent qu'il faille malgré tout gérer des rejets. Ces traitements d'exception doivent alors être spécifiés pour que les rejets soient recyclés par une procédure fonctionnelle souvent complexe. D'où l'importance de la cohérence de la transaction par rapport à l'ensemble du système.

Exemple de transaction dans le monde bancaire

Par exemple lors d'une opération informatique de transfert d'argent d'un compte bancaire sur un autre compte bancaire, il y a une tâche de retrait d'argent sur le compte source et une de dépôt sur le compte cible. Le programme informatique qui effectue cette transaction va s'assurer que les deux opérations peuvent être effectuées sans erreur, et dans ce cas, la modification deviendra alors effective sur les deux comptes. Si ce n'est pas le cas l'opération est annulée. Les deux comptes gardent leurs valeurs initiales. On garantit ainsi la cohérence des données entre les deux comptes.

Utilisation dans les bases de données

Les transactions informatiques sont très utilisées dans les SGBD.

Pseudo transactionnel

Cette ancienne technique très pratiquée avec les moniteurs transactionnels comme CICS d'IBM, TDS de BULL, UTM de Siemens, est aujourd'hui très utilisée dans les architectures des applications web, et dans les applications client-serveur.

En effet rien ne ressemble plus à une page web qu'un écran synchrone :

- la saisie est stockée en local après quelques contrôles basiques de surface ;

- et quand on tape sur « Entrée » l'ensemble des champs saisis est globalement envoyé sur le serveur central qui analyse, traite puis renvoie un nouvel écran ou une nouvelle page.

Le problème posé dans ce mode de fonctionnement est qu'il faut quelquefois un enchaînement de plusieurs écrans ou pages pour élaborer une transaction complète ACID. C'est la méthodologie Merise qui a pour la première fois défini ces concepts :

- L'enchaînement s'appelle « Procédure fonctionnelle » ;
- Le traitement entre deux écrans ou pages s'appelle « tâche ».

Cette tâche est assimilée à une pseudo-transaction qui d'un point de vue du moniteur est une transaction technique, mais bien sûr pas vraiment fonctionnelle tant que l'enchaînement n'est pas terminé.

Mais :

- Il n'est pas possible d'utiliser les mécanismes de verrouillage système pour garantir l'isolation d'un enchaînement, sinon le système s'auto-verrouille car de nombreux utilisateurs accèdent en même temps aux mêmes ressources. Les applications client/serveur utilisant les mécanismes de verrouillage des SGBD tombent souvent avec une charge de 20 utilisateurs à cause de cela.
- Il n'est pas possible de renvoyer toutes les données contextuelles déjà acquises (Utilisateur, Droits, Données déjà saisies ou lues) dans l'écran ou la page sinon les échanges deviennent trop lourds.

Questions :

- Comment garantir l'isolation d'un enchaînement de pseudo-transactions ?
- Comment gérer le contexte entre deux pseudo-transactions ?

Les réponses des anciens sont aussi celles utilisées aujourd'hui dans les « nouvelles » technologies :

- Il faut gérer un contexte par « Enchaînement » sur le serveur : dans le SGBD, un fichier ou en mémoire.
- Ce contexte doit avoir un identifiant, et une durée de vie limitée c'est-à-dire s'auto-détruire au bout d'un certain temps.
- Les objets de la base sont marqués d'une date-heure (timestamp) laquelle est modifiée à chaque mise à jour.
- Les objets nécessaires sont lus au cours des différentes tâches.
- L'information acquise utile est conservée dans le contexte : lectures, saisies, messages, notamment la DateHeure de chaque objet lu dont nous allons vérifier l'isolation a posteriori.
- Aucune mise à jour du système persistant (base de données) n'est faite en dehors de la dernière tâche.
- Lors de la dernière tâche, le programme fait ses contrôles d'intégrité fonctionnelle, et vérifie l'isolation des données en comparant la DateHeure des objets du contexte avec celle relue dans la base avant de faire la mise à jour : update where DateHeure=DateHeure-lue. Cette tâche assure les propriétés ACID de l'enchaînement.
- Évidemment, si un autre utilisateur a mis à jour un objet devant rester isolé, l'enchaînement doit être recommencé. En gestion, on peut souvent s'organiser pour que ce cas soit rare.
- S'il devait être nécessaire de réserver l'objet lors de sa lecture, il suffirait de mettre dans sa DateHeure, celle de fin de réservation, par exemple « Maintenant » + 1/4 heure. Dans

ce cas, à la lecture, il faut s'assurer que personne n'a réservé l'objet en vérifiant que sa DateHeure est bien inférieure à « Maintenant ».
- Il peut être nécessaire de contrôler qu'un objet non mis à jour n'a pas bougé. Dans ce cas, le principe reste le même : on vérifie sa DateHeure.
- Il peut être suffisant de ne contrôler qu'une donnée de l'objet. Dans ce cas, on limitera le test à cette donnée sans se soucier de la DateHeure.
- Dans une base de données, 50 % des objets sont de type « table de code » mis à jour épisodiquement par un administrateur centralisé. Il n'est pas utile de contrôler l'isolation de ces données.
- Il faut savoir évaluer le risque d'une perte d'isolation pendant l'enchaînement, et se limiter à ne contrôler que ce qui est utile.
- Dans la dernière tâche de l'enchaînement, la pseudo-transaction pose normalement et automatiquement tous les verrous système habituels garantissant l'isolation technique de l'ensemble des données mises à jour. Ce traitement ne dure que quelques centièmes de seconde, si bien que les contentions avec les autres utilisateurs sont faibles.

On comprend bien pourquoi si l'on posait des verrous système (SGBD) pendant tout l'enchaînement dont la durée est incontrôlable, le système s'écroulerait. C'est tout l'intérêt du pseudo transactionnel. Mais la stratégie du contrôle d'isolation est fondamentalement fonctionnelle.

La pseudo transaction est donc bien ACID, mais les règles fonctionnelles sont telles que la cohérence entre chaque pseudo-transaction d'un enchaînement est garantie par l'absence de mise à jour de la base de données.

Une application client/serveur bien conçue utilise aussi des pseudo-transactions, mais le contexte est géré dans l'application cliente, ce qui soulage d'autant le serveur. Le schéma type est le suivant :

- N requêtes de lectures
- Manipulations dans la mémoire de l'application cliente
- 1 requête de maj de tous les objets modifiés avec contrôle d'isolation a posteriori

Ce qui donne N+1 pseudo-transactions.

Voir aussi

- Mise à jour perdue
- Commit
- Rollback
- Moniteur transactionnel

Notes et références

1. `(en)` Philip A. Bernstein et Eric Newcomer, *Principles of transaction processing*, Morgan Kaufmann - 1997, (ISBN 9781558604155)
2. Peter McIntyre et al., *Pro PHP Programming*, Apress, page 127 : « NoSQL databases, as the name implies, are not classic SQL databases and do not implement the ACID properties. »

Bibliographie

- Jérôme Besancenot, Michèle Cart, Jean Ferrié, Rachid Guerraoui, Philippe Pucheral et Bruno Traverson, *Les Systèmes transactionnels : concepts, normes et produits*, Éd. Hermes, coll. « Informatique », octobre 1997 (ISBN 2-86601-645-9)
- **(en)** James Gray et Andreas Reuter, *Transaction Processing - Concepts and Techniques*, Morgan Kaufmann, 1993 (ISBN 1-55860-190-2)

Ce document provient de « https://fr.wikipedia.org/w/index.php?title=Transaction_informatique&oldid=235817755 ».